International University

School of Computer Science and Engineering

Artificial Intelligence

Lab 02: Vacuum Agent

Semester: 1, Year: 2025 - 2026
Lab Instructor: Dr. Ly Tu Nga

Full name: Võ Trí Khôi
Student ID: ITCSIU24045
Group: G01

1. Introduction

This report presents the implementation and evaluation of a vacuum agent that uses Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms to explore and clean an unknown grid environment. The agent starts from a random position, explores all reachable tiles, sucks up dirt, and returns to its home position at (1, 1).

1.1 Performance Scoring

The agent's performance is measured by a score with the following rules:

- Initial score: −1000
- Sucking dirt: +100 points
- Shutting down at home: +1000 points
- All other actions (forward, turn left, turn right, no-op): −1 point

1.2 GUI Preview

Here is a snapshot of the agent running in the environment, with the visual grid on the left and the action log on the right:

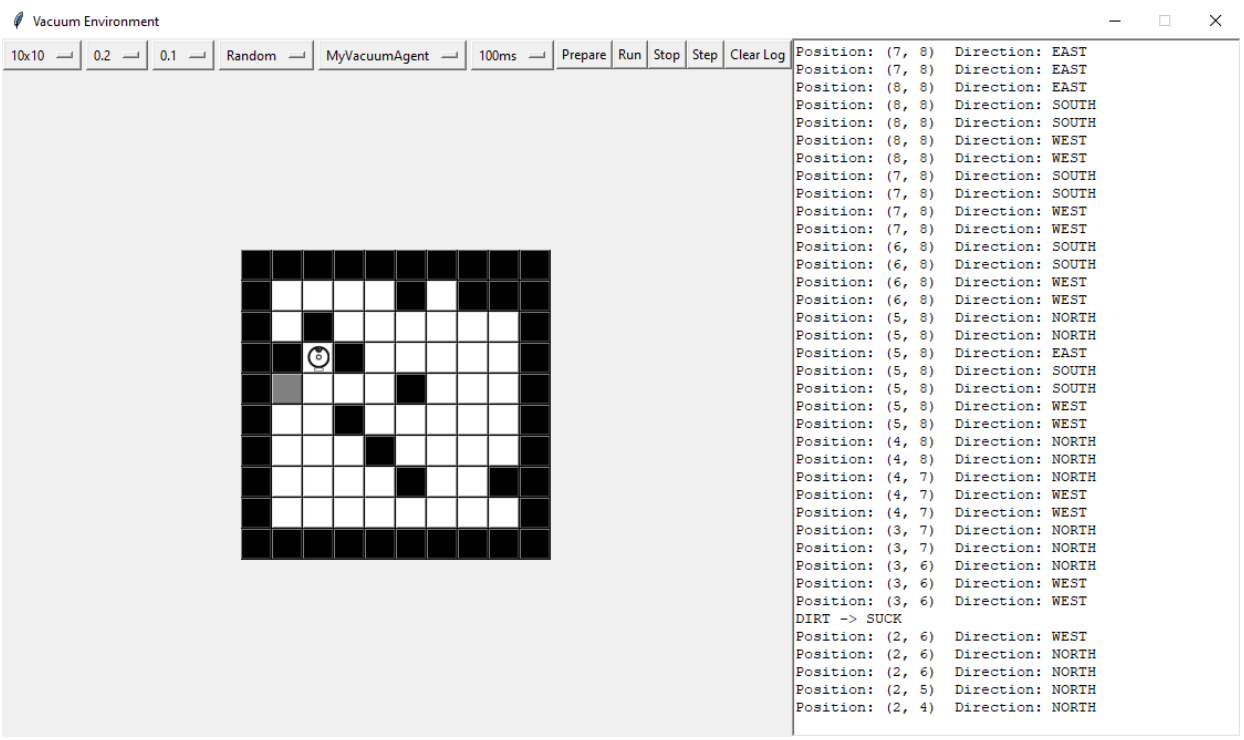

Figure 1.2.1: GUI Preview

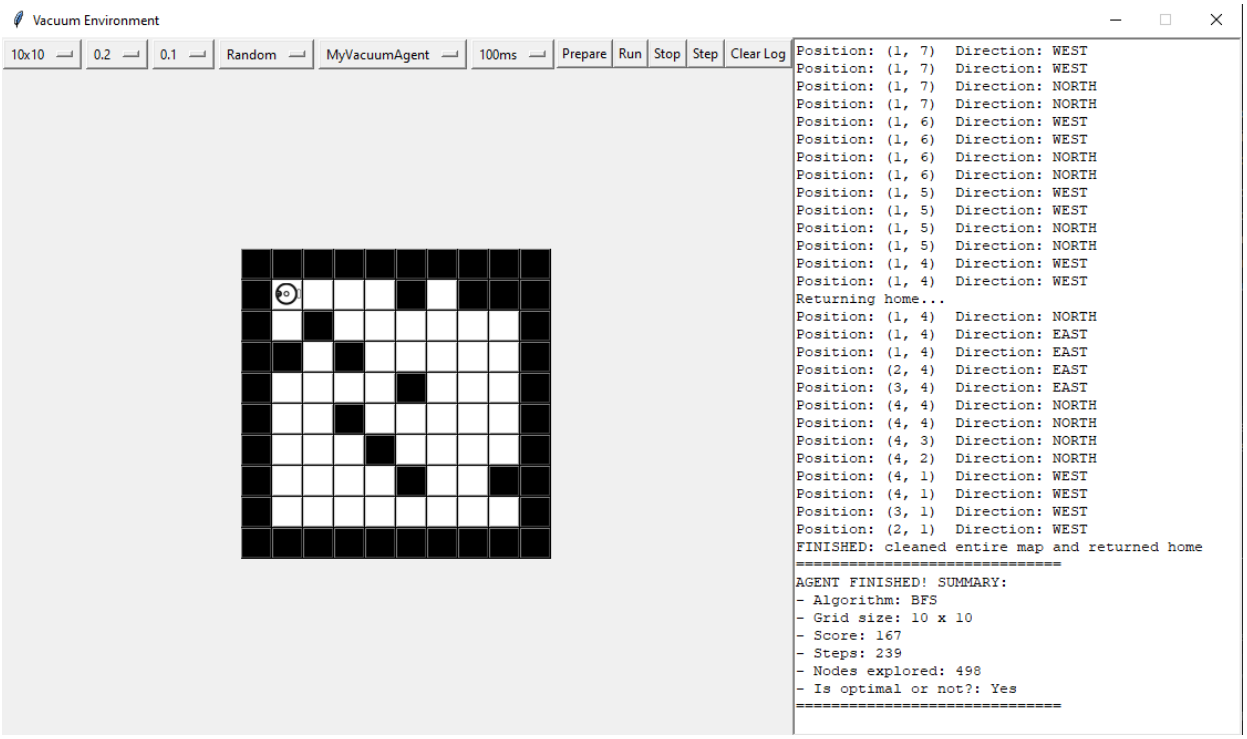

Figure 1.2.2: GUI Preview

2. Implementation

2.1 Agent Architecture

The agent is implemented in myvacuumagent.py and consists of two main classes:

- MyAgentState: Maintains the agent's internal model of the world, including a 2D grid where each cell can be UNKNOWN, WALL, CLEAR, DIRTY, or HOME.
- MyVacuumAgent: The agent itself, which uses search algorithms to decide its next action.

2.2 Agent Decision Flow

Each time the execute() method is called, the agent follows this logic:

1. Initialization Phase (first 11 steps): The agent moves to a random start position using move_to_random_start_position().
2. Perception: The agent reads percepts — bump, dirt, and home — and updates its internal world model.
3. Clean: If the current tile is dirty, the agent sucks the dirt (+100 score).
4. Plan: If no route is planned, the agent runs BFS/DFS to find the nearest unknown tile.
5. Return Home: If no unknown tiles remain, the agent uses BFS/DFS to navigate back to the home position (1, 1).
6. Shutdown: Once the agent is at home and all tiles are explored, it shuts down (+1000 score).

7. Execute Plan: If a route exists, the agent executes the next move (turn or forward).

2.3 BFS Implementation

```
    def bfs(self, return_home=False):
        # Start from current position
        start = (self.state.pos_x, self.state.pos_y)
        # Queue stores tuples of (current_position, path_to_get_there)
        queue = [(start, [])]
        visited = set()
        visited.add(start)

        while queue:
            current, path = queue.pop(0) # FIFO queue for Breadth-First Search
            cx, cy = current
            self.nodes_explored += 1

            # Goal Check
            if not return_home:
                if self.state.world[cx][cy] == AGENT_STATE_UNKNOWN:
                    self.route = path
                    return
            else:
                if current == self.home_pos:
                    self.route = path
                    return

            # Explore 4 directions (North, East, South, West)
            for dx, dy in [(0, -1), (1, 0), (0, 1), (-1, 0)]:
                nx, ny = cx + dx, cy + dy

                # Check boundaries
                if 0 <= nx < self.state.world_width and 0 <= ny <
self.state.world_height:
                    if (nx, ny) not in visited:
                        # We can walk on any tile as long as it's not a known
WALL
                        if self.state.world[nx][ny] != AGENT_STATE_WALL:
                            visited.add((nx, ny))
                            queue.append(((nx, ny), path + [(nx, ny)]))

        self.route = [] # No path found
```

How BFS works:

- BFS uses a FIFO queue (First-In, First-Out) to explore nodes level by level.

- Starting from the agent's current position, it first visits all adjacent tiles (distance 1), then all tiles at distance 2, and so on.
- It marks each tile as "visited" when it is added to the queue to prevent revisiting.
- For each node popped from the queue, it checks if the node satisfies the goal condition:
    - Exploration mode (return_home=False): The goal is any tile with AGENT_STATE_UNKNOWN.
    - Return home mode (return_home=True): The goal is the home position (1, 1).
- When a goal node is found, the path from the start to that node is stored in self.route.
- BFS guarantees the shortest path (fewest tiles) to the nearest goal because it explores nodes in order of increasing distance from the start.

2.4 DFS Implementation

```
    def dfs(self, return_home=False):
        start = (self.state.pos_x, self.state.pos_y)
        # Stack stores tuples of (current_position, path_to_get_there)
        stack = [(start, [])]
        visited = set()

        while stack:
            current, path = stack.pop() # LIFO stack for Depth-First Search
            cx, cy = current

            # In DFS, a node might be added to the stack multiple times via
different paths
            # so we mark it visited upon popping.
            if current in visited:
                continue
            visited.add(current)
            self.nodes_explored += 1

            # Goal Check
            if not return_home:
                if self.state.world[cx][cy] == AGENT_STATE_UNKNOWN:
                    self.route = path
                    return
            else:
                if current == self.home_pos:
                    self.route = path
                    return
```

```
            # Explore 4 directions (South, East, North, West)
            for dx, dy in [(0, 1), (1, 0), (0, -1), (-1, 0)]:
                nx, ny = cx + dx, cy + dy

                if 0 <= nx < self.state.world_width and 0 <= ny <
self.state.world_height:
                    if (nx, ny) not in visited:
                        if self.state.world[nx][ny] != AGENT_STATE_WALL:
                            # Push to stack
                            stack.append(((nx, ny), path + [(nx, ny)]))

        self.route = [] # No path found
```

How DFS works:

- DFS uses a LIFO stack (Last-In, First-Out) to explore nodes as deeply as possible before backtracking.
- Starting from the agent's current position, it pushes adjacent tiles onto the stack.
- When popping from the stack, it takes the most recently added node, which means it dives deep into one branch before exploring others.
- Nodes are marked as visited when popped (not when pushed), since the same node may be pushed multiple times via different paths in DFS.
- The goal check is the same as BFS — either finding an unknown tile or the home position.
- DFS does NOT guarantee the shortest path. It may find a longer, more winding path to a goal node because it explores depth-first rather than breadth-first.

2.5 Key Differences Between BFS and DFS

Feature	BFS	DFS
Data Structure	FIFO Queue	LIFO Stack
Exploration Order	Level-by-level (nearest first)	Depth-first (deepest first)
Path Optimality	Optimal (shortest path)	Not optimal
Node Visit Marking	On enqueue (push)	On dequeue (pop)
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V) — stores all nodes at frontier	O(V) — stores nodes along path

Table 2.5: Key Differences Between BFS and DFS

2.6 Move-to Logic

The move_to() method translates a path (sequence of target coordinates) into physical actions:

1. Compute target direction: Determine which direction the agent needs to face to reach the next tile.
2. Turn if necessary: If the agent is not facing the target direction, it turns right or left.
3. Move forward: Once facing the correct direction, the agent moves forward and pops the current waypoint from the route.

Each action (turn or forward) costs −1 point. Since BFS returns shorter paths, the agent needs fewer total actions, resulting in higher scores.

3. Experimental Results

All experiments were run with dirt bias = 0.5, using a fixed random seed (1337) for reproducibility.

3.1 Performance Comparison without Obstacles (wall bias = 0.0)

			BFS			DFS		
Environment size (without obstacles)	#nodes explored	Solution length (Steps)	Score	Is it optimal?	#nodes explored	Solution length (Steps)	Score	Is it optimal?
5x5	72	57	248	Yes	120	143	61	No
10x10	574	236	2190	Yes	956	10000*	9	No
15x15	1387	480	7602	Yes	2178	22500*	779	No
20x20	2359	786	14366	Yes	3978	40000*	847	No

Table 3.1: Performance Comparison without Obstacles

* DFS did not finish exploring the entire map within the iteration limit (width × height × 10). The agent ran out of iterations before it could explore all tiles and return home, resulting in significantly lower scores.

3.2 Performance Comparison with Obstacles (wall bias = 0.2)

			BFS			DFS		
Environment size (without obstacles)	#nodes explored	Solution length (Steps)	Score	Is it optimal?	#nodes explored	Solution length (Steps)	Score	Is it optimal?
5x5	83	62	243	Yes	118	140	64	No
10x10	521	247	1674	Yes	853	10000*	-193	No
15x15	1198	533	4014	Yes	2016	22500*	-534	No
20x20	2326	988	9417	Yes	2698	40000*	-1476	No

Table 3.2: Performance Comparison with Obstacles

* DFS did not finish exploring the entire map within the iteration limit.

3.3 Analysis

Nodes Explored

- BFS consistently explores fewer nodes than DFS across all grid sizes and configurations.
- With obstacles (wall bias = 0.2), both algorithms explore slightly fewer nodes on larger grids because walls reduce the number of reachable tiles. For example, on 20×20: BFS explores 2,326 nodes with walls vs 2,359 without.

Solution Length (Steps)

- BFS produces significantly shorter solutions. For a 20×20 grid without obstacles, BFS completes in 786 steps while DFS exceeds 40,000 steps (hitting the iteration limit).
- With obstacles, BFS requires more steps (988 vs 786 for 20×20) because walls force longer detours around blocked areas.

Score

- BFS achieves dramatically higher scores in all cases. For 20×20 without obstacles, BFS scores 14,366 while DFS only scores 847.
- With obstacles, scores decrease for both algorithms because there are fewer dirt tiles to clean (walls replace some dirt) and navigation requires more steps. Notably,

DFS scores become negative on larger grids with obstacles (e.g., −1,476 for 20×20), meaning the penalties outweigh the rewards.

- The score difference comes from:
  - Fewer movement actions in BFS (each costs −1 point)
  - Successful shutdown — BFS always completes and gets +1000 points, while DFS on larger grids fails to return home before running out of iterations

Impact of Obstacles

- Obstacles reduce the number of accessible tiles, which means fewer dirt tiles to clean (lower reward potential).
- Navigation becomes more complex — the agent must find paths around walls, increasing the number of turns and forward moves.
- BFS handles obstacles well since it still finds the shortest path around walls. DFS suffers more because its longer, winding paths become even worse with obstacles creating dead ends.

Optimality

- BFS is optimal: It guarantees the shortest path to any goal in an unweighted graph. Since all edges have equal cost (1 action = 1 step), BFS always finds the path with the minimum number of actions.
- DFS is NOT optimal: It does not guarantee the shortest path. DFS explores one branch completely before trying alternatives, which can lead to unnecessarily long paths.

4. Conclusion

The experimental results clearly demonstrate that BFS outperforms DFS for the vacuum agent problem across all metrics, both with and without obstacles:

1. Fewer nodes explored — BFS searches more efficiently by expanding level-by-level.
2. Shorter solution paths — BFS guarantees shortest paths, leading to fewer total actions.
3. Higher scores — Fewer actions mean fewer penalties, and successful completion yields the +1000 shutdown bonus.
4. Optimality — BFS is provably optimal for shortest-path finding in unweighted graphs, while DFS is not.
5. Robustness — BFS maintains strong performance even with obstacles, while DFS degrades severely (negative scores on large grids with walls).

The key takeaway is that for problems where finding the nearest goal is important (like finding the nearest unexplored tile), BFS is the superior choice. DFS, while theoretically having the same time complexity, performs poorly in practice because it cannot guarantee finding nearby goals first.